DATA STRUCTURES IN PYTHON

Internship Assignment – Week 3

Submitted By

Name: Amrutha B V
Course: Vth Semester BCA
Subject: Data Structures in Python

ACKNOWLEDGEMENT:-

I would like to express my sincere gratitude to my internship mentors and faculty members for providing me with the opportunity to complete this assignment on Data Structures in Python. This assignment helped me understand how different data structures are created, accessed, modified and managed in Python.

I am also thankful for the guidance and support provided throughout the internship. This assignment improved my practical knowledge of Lists, Tuples, Sets and Dictionaries.

1. INTRODUCTION:-

Data structures are used to store and organize data efficiently in a program. Python provides several built-in data structures such as Lists, Tuples, Sets and Dictionaries.
Each data structure has different features and uses. Lists are ordered and mutable, Tuples are ordered and immutable, Sets store unique values, and Dictionaries store data in key-value pairs.
The objective of this assignment is to understand these basic data structures and learn how to create, access, modify and manage multiple values using Python.

2. OBJECTIVE:-

The main objectives of this assignment are:

- To understand basic data structures in Python.
- To learn how to create and access Lists, Tuples, Sets and Dictionaries.
- To perform different operations on data structures.
- To understand indexing and slicing.
- To learn commonly used data structure methods.
- To understand how data structures can be used with loops.
- To develop practical programming skills in Python.

3. TOPICS COVERED:-

The following topics are covered in this assignment:

1. Lists
2. Tuples
3. Sets
4. Dictionaries
5. Indexing and Slicing
6. Common Data Structure Methods
7. Nested Data Structures
8. Loops with Data Structures

TASK 1: LISTS

Aim:-
To create a Python program using a List and demonstrate creating, accessing, adding, updating, removing and sorting elements.

Introduction:-
A List is an ordered and changeable collection in Python. Lists can contain multiple values of different data types. Elements in a list can be accessed using indexing.

Lists are created using square brackets "[ ]".

Operations Demonstrated

- Creating a list
- Accessing elements
- Adding elements
- Updating elements
- Removing elements
- Sorting the list

Explanation:-

Creating a List:-
A list is created by placing elements inside square brackets.

Accessing Elements:-
List elements can be accessed using their index position. Python uses zero-based indexing.

Adding Elements:-
The "append()" method is used to add an element at the end of a list.

Updating Elements:-

An existing element can be changed by assigning a new value to its index.

Removing Elements:-
The "remove()" method is used to remove a specific element from a list.

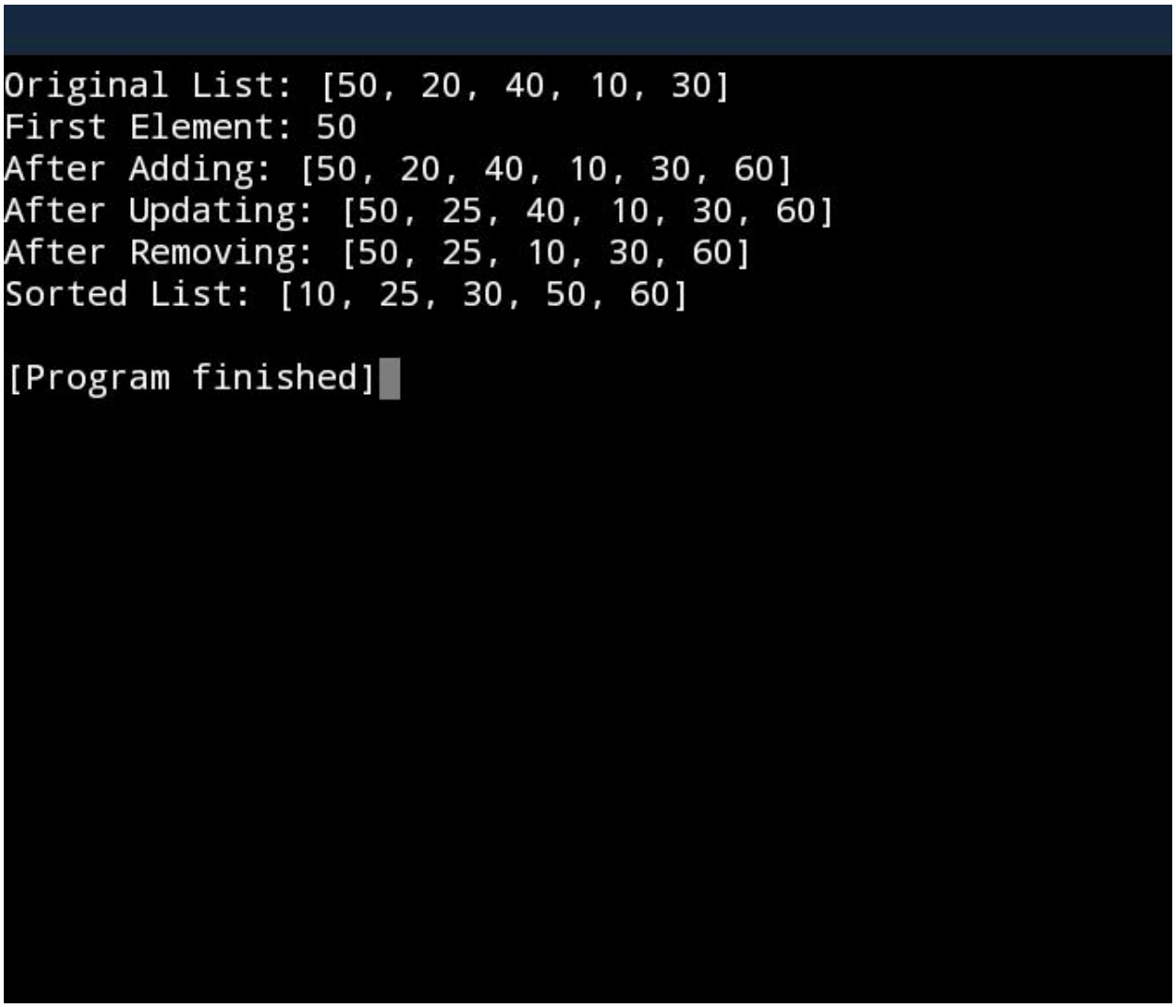
Sorting the List:-
The "sort()" method arranges the elements in ascending order.

Source Code:-

new*

```python
# Task 1: Lists

numbers = [50, 20, 40, 10, 30]

print("Original List:", numbers)

# Accessing element
print("First Element:", numbers[0])

# Adding element
numbers.append(60)
print("After Adding:", numbers)

# Updating element
numbers[1] = 25
print("After Updating:", numbers)

# Removing element
numbers.remove(40)
print("After Removing:", numbers)

# Sorting list
numbers.sort()
print("Sorted List:", numbers)
```

Expected Output:-
The program displays the original list, accessed element, updated list, list after adding and removing elements, and the sorted list.



Learning Outcome:-
After completing this task, I learned how to create and manipulate lists in Python. I also learned how to access, add, update, remove and sort list elements.

-TASK 2: TUPLES
Aim:-
To create a Python program using Tuples and demonstrate indexing, slicing, length and basic tuple operations.

Introduction:-

A Tuple is an ordered collection of elements in Python. Tuples are immutable, which means their elements cannot be changed after creation.

Tuples are created using parentheses "( )".

Operations Demonstrated

- Creating a tuple
- Accessing elements using indexing
- Tuple slicing
- Finding the length
- Basic tuple operations

Explanation:-
Creating a Tuple:-
A tuple is created by placing elements inside parentheses.

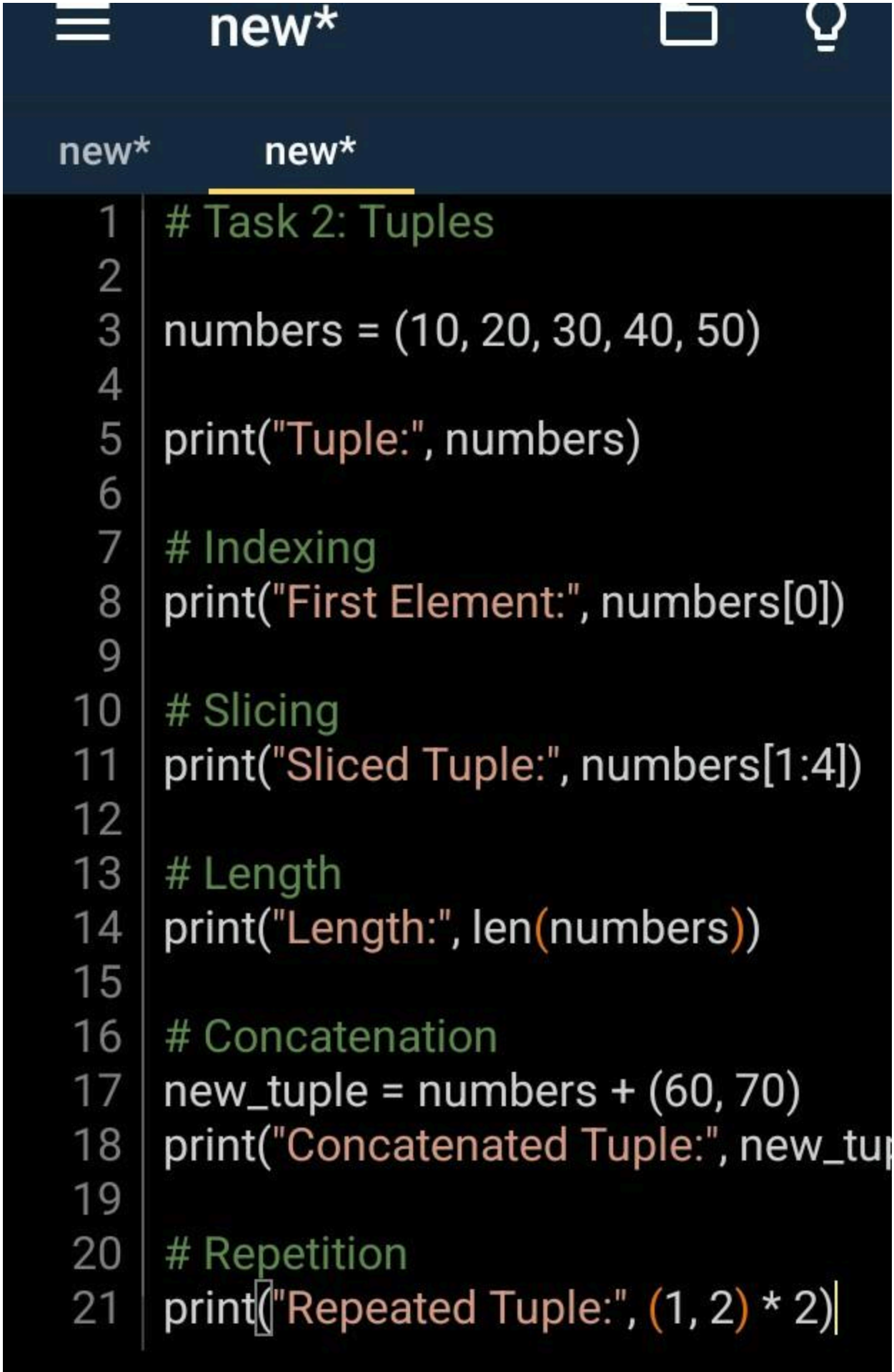
Indexing:-
Individual tuple elements can be accessed using their index number.

Tuple Slicing:-
Slicing is used to access a range of elements from a tuple.

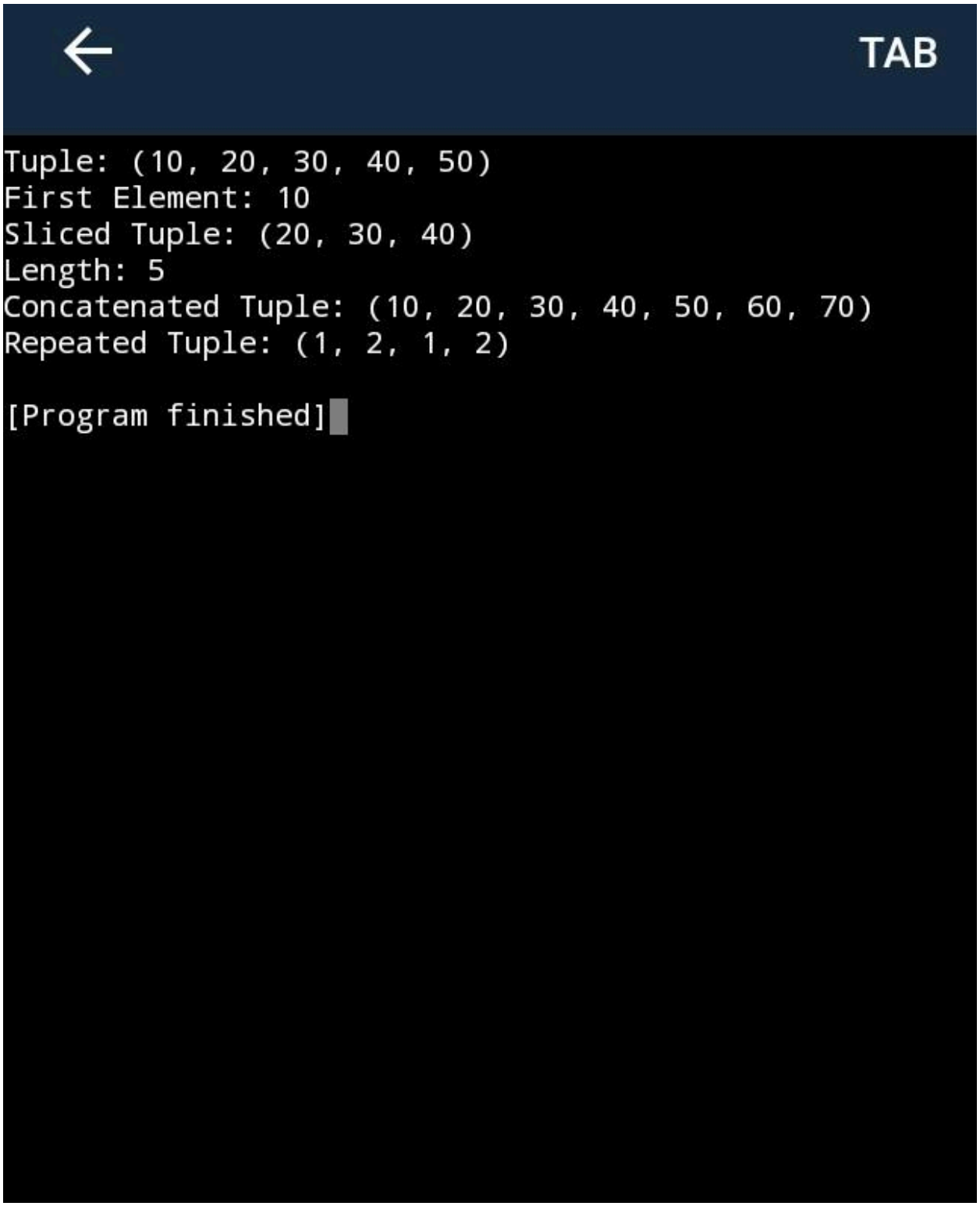
Finding Length:-
The "len()" function returns the number of elements in a tuple.

Basic Tuple Operations:-
Tuples support operations such as concatenation and repetition.

Source Code:-


new*
new*
new*
1 # Task 2: Tuples
2
3 numbers = (10, 20, 30, 40, 50)
4
5 print("Tuple:", numbers)
6
7 # Indexing
8 print("First Element:", numbers[0])
9
10 # Slicing
11 print("Sliced Tuple:", numbers[1:4])
12
13 # Length
14 print("Length:", len(numbers))
15
16 # Concatenation
17 new_tuple = numbers + (60, 70)
18 print("Concatenated Tuple:", new_tup
19
20 # Repetition
21 print("Repeated Tuple:", (1, 2) * 2)

Expected Output:-



The program displays the tuple, accessed element, sliced tuple, length of the tuple, concatenated tuple and repeated tuple.

Learning Outcome

After completing this task, I learned how tuples work in Python and understood indexing, slicing, length and basic tuple operations.

TASK 3: SETS

Aim
To create a Python program using Sets and demonstrate removing duplicate values, adding and removing elements, union, intersection and difference.

Introduction:-
A Set is an unordered collection of unique elements in Python. Sets automatically remove duplicate values.

Sets are created using curly brackets "{ }" or the "set()" function.

Operations Demonstrated

- Creating a set
- Removing duplicate values
- Adding elements
- Removing elements
- Union
- Intersection
- Difference

Explanation:-
Creating a Set:-
A set is created by placing unique elements inside curly brackets.

Removing Duplicate Values:-
Sets automatically remove duplicate values.

Adding Elements:-
The "add()" method is used to add a new element to a set.

Removing Elements:-
The "remove()" method can be used to remove an element from a set.

Union:-
Union combines all unique elements from two sets.

Intersection:-

Intersection returns only the common elements between two sets.

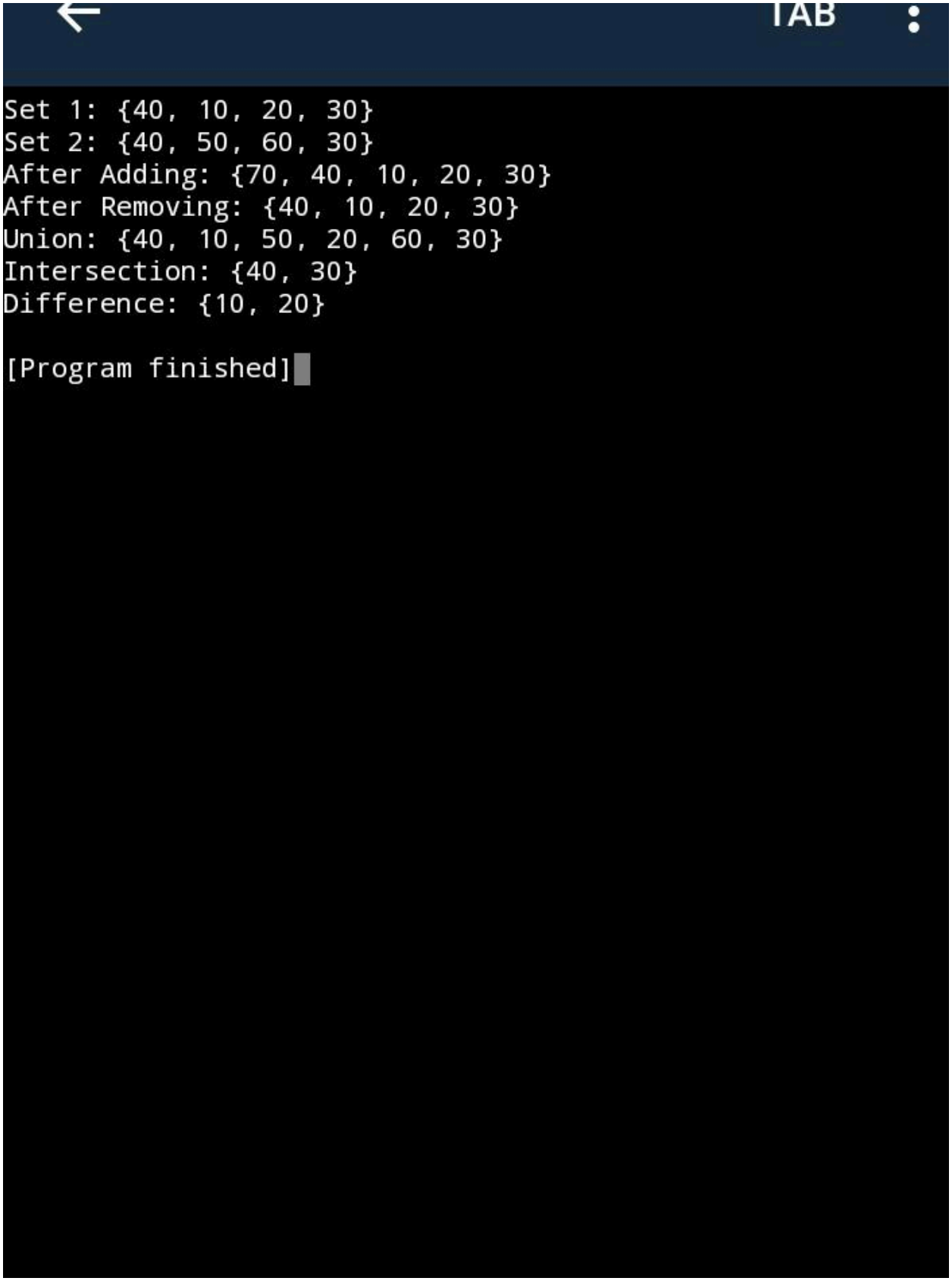
Difference:-
Difference returns the elements that are present in one set but not in the other.

Source Code:-

new*

```python
# Task 3: Sets

set1 = {10, 20, 20, 30, 40}
set2 = {30, 40, 50, 60}

print("Set 1:", set1)
print("Set 2:", set2)

# Adding element
set1.add(70)
print("After Adding:", set1)

# Removing element
set1.remove(70)
print("After Removing:", set1)

# Union
print("Union:", set1.union(set2))

# Intersection
print("Intersection:", set1.
intersection(set2))

# Difference
print("Difference:", set1.difference(set2))
```

Expected Output:-


TAB
Set 1: {40, 10, 20, 30}
Set 2: {40, 50, 60, 30}
After Adding: {70, 40, 10, 20, 30}
After Removing: {40, 10, 20, 30}
Union: {40, 10, 50, 20, 60, 30}
Intersection: {40, 30}
Difference: {10, 20}
[Program finished]

The program displays the set after removing duplicates, added and removed elements, union, intersection and difference results.

Learning Outcome:-

After completing this task, I learned how sets store unique values and how to perform union, intersection and difference operations.

TASK 4: DICTIONARIES

Aim:-
To create a Student Information Dictionary and demonstrate accessing, adding, updating, removing and displaying key-value pairs.

Introduction:-
A Dictionary is a collection of data stored in the form of key-value pairs. Each key is used to access its corresponding value.

Dictionaries are created using curly brackets "{ }".

For example:

"{"Name": "Amrutha", "Age": 19}"

Here, "Name" and "Age" are keys and "Amrutha" and "19" are their corresponding values.

Student Information:-

The dictionary contains the following details:

- Name
- Age
- Course
- City

Operations Demonstrated

- Creating a dictionary
- Accessing values
- Adding new key-value pairs
- Updating values
- Removing data
- Displaying keys and values
- Using dictionary methods

Explanation:-

Accessing Values:-
Values can be accessed using their corresponding keys.

Adding New Key-Value Pairs:-
A new key and value can be added by assigning a value to a new key.

Updating Values:-
An existing value can be changed by assigning a new value to its key.

Removing Data:-
The "pop()" method can be used to remove a key-value pair from a dictionary.

Displaying Keys and Values:-
The "keys()" and "values()" methods are used to display dictionary keys and values.

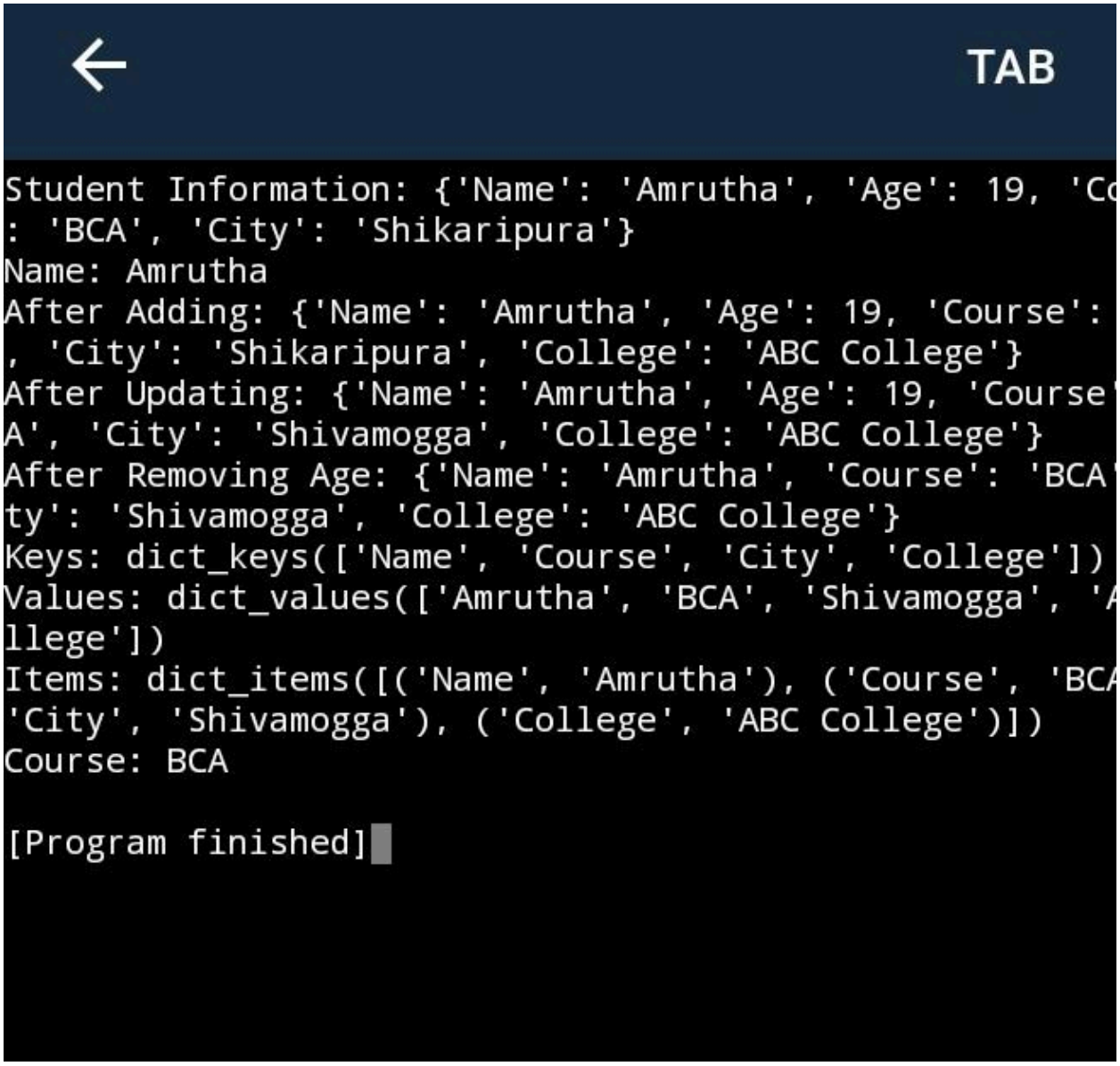
Dictionary Methods:-
Common dictionary methods include:

- "keys()"
- "values()"
- "items()"
- "get()"
- "pop()"

Source Code:-

```python
# Task 4: Student Information Dictionary

student = {
    "Name": "Amrutha",
    "Age": 19,
    "Course": "BCA",
    "City": "Shikaripura"
}
# Display dictionary
print("Student Information:", student)
# Accessing value
print("Name:", student["Name"])
# Adding new key-value pair
student["College"] = "ABC College"
print("After Adding:", student)
# Updating value
student["City"] = "Shivamogga"
print("After Updating:", student)
# Removing data
student.pop("Age")
print("After Removing Age:", student)
# Displaying keys
print("Keys:", student.keys())

# Displaying values
print("Values:", student.values())
```

Expected Output:-



The program displays student information, accesses the student's name, adds a new key-value pair, updates an existing value, removes data and displays keys and values.

Learning Outcome:-

After completing this task, I learned how dictionaries store information using key-value pairs. I also learned how to access, add, update, remove and display dictionary data using different methods.

4. INDEXING AND SLICING:-

Indexing is used to access an individual element from a sequence. Python uses zero-based indexing, where the first element has index "0".

Slicing is used to access a range of elements from a sequence.

Example:

```
numbers = [10, 20, 30, 40, 50]

print(numbers[0])
print(numbers[1:4])
```

Indexing accesses one element, while slicing accesses multiple elements.

5. COMMON DATA STRUCTURE METHODS:-

Python provides many built-in methods for working with data structures.
Some commonly used methods are:

- "append()" – adds an element to a list.
- "remove()" – removes an element from a list or set.
- "sort()" – sorts list elements.
- "add()" – adds an element to a set.
- "union()" – combines two sets.
- "intersection()" – finds common elements.
- "keys()" – returns dictionary keys.
- "values()" – returns dictionary values.
- "items()" – returns key-value pairs.
- "pop()" – removes an element.

6. NESTED DATA STRUCTURES:-

A nested data structure means one data structure is stored inside another data structure.
For example, a dictionary can contain a list:

```
student = {
    "Name": "Amrutha",
    "Marks": [80, 85, 90]
}
```

Nested data structures are useful for storing complex information in an organized manner.

7. LOOPS WITH DATA STRUCTURES:-

Loops can be used to access and display multiple elements from data structures.
For example:

```
students = ["Amrutha", "Anu", "Priya"]

for student in students:
    print(student)
```

The "for" loop accesses each element one by one.

9. CONCLUSION:-

This assignment helped me understand the basic data structures available in Python. I learned how to create, access and modify Lists, Tuples, Sets and Dictionaries.

I also learned important operations such as indexing, slicing, adding, updating, removing, sorting, union, intersection, difference and dictionary methods.

The practical programs helped me improve my understanding of how data can be stored and managed efficiently in Python. Overall, this assignment improved my programming skills and provided a strong foundation for working with Python data structures.

10. OVERALL LEARNING OUTCOME:-

After completing this assignment, I am able to:

- Create and use Lists in Python.
- Perform different List operations.
- Create and access Tuples.
- Perform tuple indexing and slicing.
- Create Sets and remove duplicate values.
- Perform set operations such as union, intersection and difference.
- Create Dictionaries using key-value pairs.
- Add, update and remove dictionary data.
- Use common data structure methods.
- Work with nested data structures and loops.
- Understand the practical use of Python data structures.